

Author
Christian Kühberger
11919229

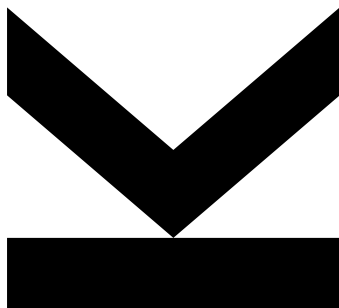
Submission
Institute for Machine
Learning

Thesis Supervisor
Univ.-Prof. Dr.
Sepp Hochreiter

Assistant Thesis Supervisor
Dr. **Fabian Paischer**, MSc

March 2026

xLSTM-Based Memory for Semantic History Compression in Reinforcement Learning



Bachelor's Thesis

to confer the academic degree of

Bachelor of Science

in the Bachelor's Program

Artificial Intelligence

Abstract

Semantic History Compression via Language Models (SHELM) is a reinforcement learning architecture designed to handle partially observable environments by summarizing past observations in natural language. It combines semantic encoding with a Language Model (LM) memory module to produce a compressed history representation that is used as input for the policy that chooses the next action.

While SHELM achieves strong empirical performance in environments that require a memory component to solve, its use of a TransformerXL (TrXL) memory module results in quadratic computational complexity, which limits scalability as the context grows. In this thesis, we replace the TrXL memory module with the Extended Long Short-Term Memory (xLSTM) architecture and integrate the 7-billion parameter xLSTM-7B model as an alternative recurrent memory. xLSTM is chosen for its recurrent design and linear computational complexity, making it a suitable alternative to attention-based memory for long contexts.

To enable the use of the large xLSTM model under hardware constraints, 8-bit quantization is applied to the model weights. The remainder of the SHELM pipeline, including the CLIP-based semantic encoding, is left unchanged to isolate the effect of the memory replacement. We evaluate the resulting system on the MiniGrid-Memory benchmark and show that replacing the Transformer-XL with a quantized xLSTM-7B backbone improves sample efficiency by 3x. Evaluated on the Psychlab Continuous Recognition task the xLSTM also outperforms the original implementation over 6 million timesteps.

Contents

Abstract	ii
1 Introduction	1
2 Background	3
2.1 Reinforcement Learning Foundations	3
2.1.1 Markov Decision Processes (MDP)	4
2.1.2 Partial Observability (POMDP)	4
2.2 History Compression via Language Models in Reinforcement Learning . .	5
2.2.1 History Compression via Language Models (HELM)	5
2.2.2 HELMv2	5
2.2.3 Semantic History Compression (SHELM)	5
2.2.4 The Role of the Language Model	6
2.3 Sequence Modeling Architectures	7
2.3.1 Transformers and Transformer-XL	7
2.3.2 Long Short-Term Memory (LSTM)	7
2.3.3 Extended Long Short-Term Memory (xLSTM)	8
2.4 Large-Scale Model Optimization	9
2.4.1 Quantization	9
3 Experiments	11
3.1 Methodology - Architecture Integration	11
3.2 MiniGrid-Memory	12
3.3 PsychLab - Continuous Recognition	12
4 Results	14
4.1 Minigrid-Memory Results	14
4.1.1 Optimization Stability Analysis	14
4.1.2 Comparative Benchmark	15
4.1.3 Effect of Layer Normalization on History Compression	16
4.2 PsychLab - Continuous Recognition Results	16
4.2.1 Learning Performance	16
5 Discussion	19
5.1 Interpretation of Performance Gains	19
5.2 Asymptotic vs. Practical Runtime	20
5.3 Optimization Dynamics and Stability	21
5.4 Limitations and Future Work	21
6 Conclusion and Outlook	23
Bibliography	25
List of Figures	27
List of Tables	28

Chapter 1

Introduction

Sequential decision-making problems where an agent interacts to maximize rewards can be tackled using Reinforcement Learning (RL) [25]. In the real world, many of these RL problems can be formulated as Partially Observable Markov Decision Processes (POMDPs). They are "Partially Observable" because parts of true state of the world are hidden from the agent. In contrast, a Markov Decision Process (MDP) is the standard formulation where the agent has access to all necessary information to optimally solve the task.

To solve POMDPs, an agent must handle the observed incomplete state relying on both current information and all previous steps. This is achieved by storing past observations in a memory module, which is then used to decide the next action. Directly storing the entire history of raw observations is computationally intractable as the time horizon extends. Consequently, agents must rely on history compression to map long-range dependencies into a fixed-size representation [23]. Common methods for this include LSTMs [13], Transformers [27], and world models such as Dreamer [11].

One novel approach to solving these POMDPs is History ComprESSION via Language Models (HELM) [20]. Natural language has always been a vital way for humans to compress and convey information. This architecture utilizes a pretrained Language Model (LM) to abstract past observations into the LM's embedding space, creating a compressed history representation. This memory is then used by an actor-critic network alongside the current observation to solve the task.

An update to this architecture led to Semantic HELM (SHELM) [19], which introduced a human-readable memory and also improved the performance of the framework in environments which require a memory component. SHELM uses Contrastive Language-Image Pre-Training (CLIP) [22] to tokenize visual inputs into language tokens. A shared vocabulary between CLIP and the LM is established by mapping the visual features identified by CLIP to the input vocabulary of the LM. The LM then acts as a memory module by storing these tokens in its cache, forming a compressed history of all preceding events.

The original SHELM architecture utilizes Transformer-XL (TrXL) [6] as its core memory module. Standard Transformers are usually limited by their fixed-length segments, so TrXL was specifically chosen since it captures longer temporal dependencies through a segment-level recurrence mechanism. Despite these improvements in context length, the underlying architecture still relies on global self-attention. This results in a computational complexity of $O(N^2)$ relative to the sequence length N , creating a significant performance bottleneck as episode lengths increase in real-time RL tasks.

Previous optimizations of SHELM [12, 18] have mainly focused on model compression, such as distillation or pruning. Although effective in lowering resource requirements, these methods reduce model capacity and may limit the expressiveness of the learned memory

representation. This thesis addresses the research gap of whether we can achieve linear scaling $O(N)$ while maintaining a massive parameter count.

The recently developed xLSTM-7B [4] is an ideal candidate for this use case, as it combines linear complexity with a Large Language Model capacity of 7 billion parameters. By replacing the global attention mechanism with mLSTM (matrix memory) and sLSTM (scalar memory), xLSTM achieves a constant-size state update. This transition from global self-attention to linear recurrence is what enables 7B-parameter capacity to remain efficient over long horizons.

In this work, we compare the original SHELM pipeline (using Transformer-XL) with a modified version using a quantized xLSTM-7B model. The framework is evaluated on the MiniGrid-Memory benchmark [5], which requires a robust memory component for success. We show that replacing the Transformer-XL with a quantized xLSTM-7B backbone improves sample efficiency by 3x. Specifically, the xLSTM agent reached a solved state in 500,000 timesteps, compared to the 1.4 million steps required by the 18-layer Transformer baseline. Furthermore, the model maintains competitive performance on the visually complex PsychLab Continuous Recognition task beating out the Transformer-XL over 6 million timesteps and demonstrating the robustness of large-scale recurrent memory for long-horizon reinforcement learning.

Chapter 2

Background

This chapter reviews the fundamental techniques and theoretical background required to understand the experiments presented later in this thesis. We begin with the foundations of Reinforcement Learning (RL), specifically Markov Decision Processes (MDPs) and their partially observable counterparts (POMDPs), which form the backbone of our problem setting. We then discuss the Semantic History Compression via Language Models (SHELM) architecture, detailing its evolution and the critical role the Language Model plays as a memory module. Finally, we examine different sequence modeling architectures, with a special focus on the Extended Long Short-Term Memory (xLSTM) used in this work, and conclude with a brief overview of Quantization, the optimization technique enabling the deployment of large-scale models.

2.1 Reinforcement Learning Foundations

Reinforcement Learning is a computational framework for learning goal-directed behavior through interaction with an environment. Sutton and Barto [25] identify the following key elements that characterize a Reinforcement Learning system:

Agent: The learner and decision-maker in the system. The agent interacts with the environment by selecting actions based on its observations.

Environment: The external system with which the agent interacts continuously. Upon receiving an action from the agent, the environment transitions to a new state and returns a numerical reward signal. The agent’s objective is to maximize the cumulative reward received from the environment over time.

Policy: The policy maps the current state of the environment to the actions of the agent. It defines the agent’s behavior at any given time, dictating how the agent acts in response to the signals received from the environment.

Reward Signal: The reward signal defines the goal of the reinforcement learning problem. At each time step, the environment sends the agent a single number, the reward, which provides immediate feedback on the desirability of the agent’s action.

Value Function: While the reward signal indicates what is good in the immediate sense, the value function specifies what is good in the long run. It estimates the total amount of reward an agent can expect to accumulate starting from a specific state. A state might yield a low immediate reward but have a high value because it is followed by other states that yield high rewards.

And in some cases a **World Model:** This component approximates the transition dynamics of the environment. It allows the agent to predict how the environment will respond to its actions (e.g., predicting the next state and reward). Models are used for

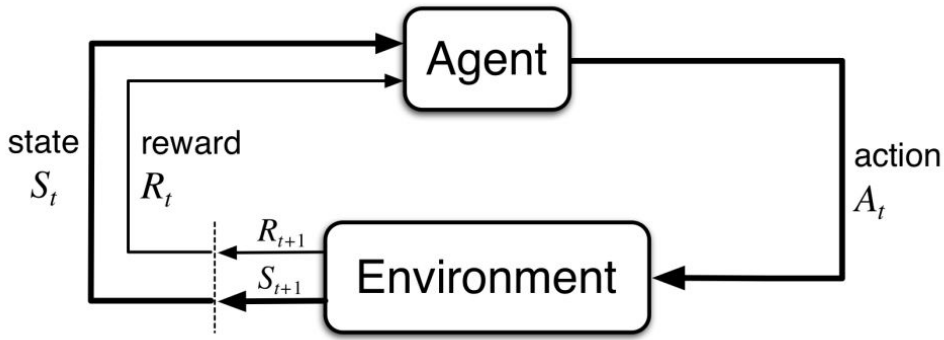


Figure 2.1: Illustration of the standard Reinforcement Learning interaction loop [25].

planning, enabling the agent to decide on a course of action by considering possible future situations before they are experienced. The standard Reinforcement Learning loop with an Agent, Environment and a state, action and a following reward is shown in 2.1.

2.1.1 Markov Decision Processes (MDP)

A sequential decision making problem is typically formalized as a Markov Decision Process (MDP) [25]. An MDP is defined by the tuple (S, A, R, P, γ) , where S represents the set of finite states, A the set of finite actions, R the reward function, P the transition probability function and the final element, $\gamma \in [0, 1]$, is the discount factor.

The state representation S_t contains all necessary information to make an optimal decision, a characteristic known as the *Markov property* [25]. This implies that the probability of the next state and reward depends only on the current state and action, not on the history of earlier states.

2.1.2 Partial Observability (POMDP)

In many real-world scenarios—and specifically in the benchmarks used in this thesis—the agent does not have access to the full underlying state of the environment. This scenario is formalized as a Partially Observable Markov Decision Process (POMDP) [15].

A POMDP can be viewed as an extension of a standard MDP and is described by the extended tuple $\langle S, A, P, R, \Omega, O, \gamma \rangle$, where S, A, P, R and γ are analogous to the definitions in section 2.1.1.

In a POMDP, the agent does not observe the state S_t directly; instead, it receives an observation $O_t \in \Omega$ which provides only a partial view of the true state. The observation function $Z(o|s', a)$ defines the probability distribution over possible observations o , given that the agent took action a and the environment transitioned to state s' .

Because the current observation is insufficient to determine the true state, the agent must aggregate past observations into a memory or belief state to infer the underlying state of the environment and act optimally.

2.2 History Compression via Language Models in Reinforcement Learning

This section outlines the evolution of using pre-trained Language Models for memory compression in Reinforcement Learning, detailing the progression from the original HELM architecture to Semantic HELM (SHELM).

2.2.1 History Compression via Language Models (HELM)

The original implementation [20] introduced using a pre-trained LM as a memory module. It utilized a mechanism called *FrozenHopfield* (FH) to map visual observations into the LM’s token embedding space. The FH mechanism performs a randomized attention over pre-trained token embeddings using a fixed random projection matrix, effectively retrieving a mixture of tokens without requiring training. A major drawback was the representation collapse of visually similar observations, where distinct states led to identical token embeddings, preventing the agent from distinguishing between different objects.

2.2.2 HELMv2

Contrastive Language-Image Pre-Training (CLIP) [22] is a neural network trained on millions of image-text pairs to understand how visual concepts relate to language. Specifically, it employs a dual-encoder architecture to project both visual inputs and their accompanying textual descriptions into a shared, multimodal latent space. By utilizing a contrastive loss function, the model is optimized to maximize the cosine similarity between the embeddings of matching image-text pairs while simultaneously minimizing the similarity of mismatched combinations within a given batch.

To combat this problem the next iteration of HELM [21] replaced the random projections of the FrozenHopfield with a pre-trained CLIP encoder to capture visual semantics. It also aligned CLIP visual embeddings with the LM space using a batch normalization layer with fixed shifting and scaling parameters. But still there was no semantic meaning transported from CLIP to the Language Model in the Memory component. CLIP also struggled with converting simplistic 2D environments into semantic meaning.

2.2.3 Semantic History Compression (SHELM)

SHELM [19] resolved these issues by introducing a discrete retrieval mechanism that maps images to semantically correct, discrete words rather than continuous embeddings. Semantic consistency is maintained by CLIP aligning the text and image spaces in its embedding space. These changes significantly improved performance in memory-intensive tasks and had the great side effect that it now provided a human-readable memory trace, allowing researchers to verify if crucial information was stored.

The workflow of SHELM operates by bridging the visual and textual domains. First, a semantic database S is constructed by identifying the set of overlapping tokens present in the vocabularies of both the Language Model and the CLIP text tokenizer. This pre-alignment ensures that any textual description retrieved based on visual features is within the Language Model’s known distribution. During the agent’s interaction, the CLIP vision encoder processes the current observation o_t , and the system selects the top- k tokens from the database S via cosine similarity.

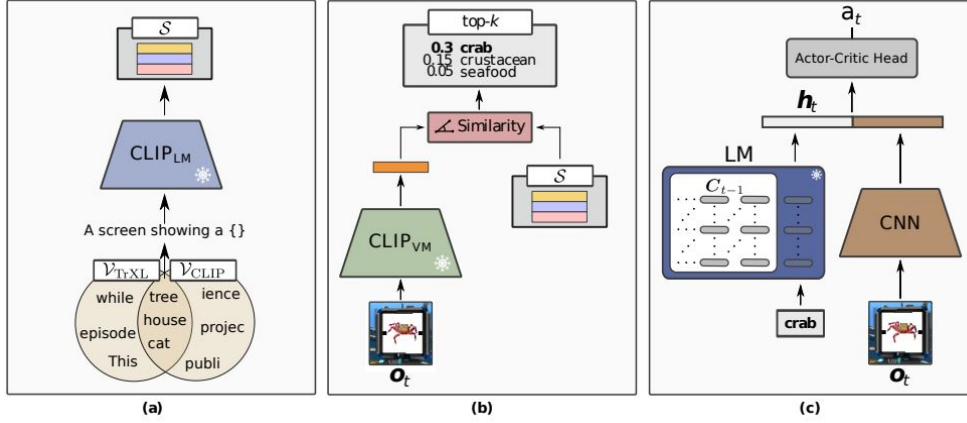


Figure 2.2: Overview of the Semantic HELM (SHELM) architecture [19].

These semantically meaningful tokens, which represent textual descriptions of the observation, are then fed into the Language Model to form the memory component. Simultaneously, a **Convolutional Neural Network (CNN)** [16] extracts immediate spatial features from the current observation. A CNN is a deep learning architecture specifically designed for processing regular grid-like data, such as images, by using filters to detect patterns like edges and shapes. The agent’s policy combines the compressed history h_t from the LM and the spatial features from the CNN to select an action a_t . Figure 2.2 gives a high-level overview over the SHELM architecture.

The CNN and policy head are trained using **Proximal Policy Optimization (PPO)** [24], a reinforcement learning algorithm that improves the agent’s policy iteratively. During training, the parameters of the CNN as well as the policy and value function are adjusted based on the received reward to maximize performance, while the LM remains a stable, pre-trained anchor.

2.2.4 The Role of the Language Model

The Language Model (LM) functions as the central engine for history compression in SHELM. Its primary role is to act as a recurrent memory cache that processes the flow of textual descriptions provided by the retrieval system.

By ingesting the sequence of discrete tokens, the LM builds an internal hidden state h_t that summarizes the agent’s entire experience within an episode. This allows the agent to make decisions based on long-term dependencies without the need to store raw visual data. In earlier versions of the framework, this role was fulfilled by a TransformerXL [6]. While the TransformerXL provides high-capacity recall, its self-attention mechanism incurs a quadratic computational cost $\mathcal{O}(N^2)$ relative to the sequence length N . This scaling issue provides the primary motivation for this thesis, which investigates replacing the Transformer-based memory with the more efficient xLSTM architecture.

2.3 Sequence Modeling Architectures

The capacity of an agent to store and recall past events is limited by the architecture of its memory module. In the context of Semantic History Compression, this module must process long sequences of text tokens efficiently. While the Transformer has become the standard for such tasks, its computational overhead at long sequences has led to a renewed interest in recurrent models like the xLSTM.

2.3.1 Transformers and Transformer-XL

The Transformer architecture, introduced by Vaswani et al. [27], shifted the paradigm of sequence modeling from recurrence to self-attention. This allows the model to process all tokens in a sequence simultaneously rather than one by one. The core of this architecture is the *Scaled Dot-Product Attention*.

In this mechanism, the input is transformed into three vectors: Queries (Q), Keys (K), and Values (V). The model calculates the compatibility between a Query and all Keys to determine how much "attention" to pay to each corresponding Value. This is expressed mathematically as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.1)$$

The factor $\sqrt{d_k}$ is used to scale the dot product and prevent gradients from becoming too small. While powerful, standard Transformers use a fixed-length context window. Once the sequence exceeds this length, older information is discarded, which can lead to context fragmentation.

To solve this, the Transformer-XL (TrXL) [6] was developed. It introduces segment-level recurrence, where the hidden states of the previous segment are cached and reused as context for the current segment. Additionally, it uses relative positional encodings so the model can understand the distance between tokens across different segments. Despite these improvements in context length, the underlying attention mechanism still requires computing a similarity matrix for all token pairs. Consequently, TrXL still suffers from a quadratic complexity of $\mathcal{O}(N^2)$, meaning the computational cost grows quadratically as the sequence length N increases.

2.3.2 Long Short-Term Memory (LSTM)

Before Transformers became the standard, the Long Short-Term Memory (LSTM) [13] was the primary tool for sequence modeling. It was designed to solve the vanishing gradient problem, a common issue in deep learning networks where the gradients used for learning would become too small, causing the model to forget information from the beginning of a sequence.

The LSTM addresses this by using a mechanism called a constant error carousel, which is implemented through the cell state. This state acts as a long-term memory track that allows information to flow backward through time without vanishing. The original architecture manages this track using two gates: the input gate, which controls the addition of new observations to the memory, and the output gate, which determines what parts of the memory are exposed to the rest of the network.

Later a forget gate was introduced [9] to allow the model to reset its state and discard past information, but potentially re-introducing the vanishing gradient problem if not carefully

initialized. Because the LSTM processes inputs sequentially across discrete time steps while maintaining a fixed-size state, its computational complexity is linear, denoted as $\mathcal{O}(N)$. However, traditional LSTMs struggle to match the performance of modern models because they cannot easily revise their memory and are slow to train on large clusters.

2.3.3 Extended Long Short-Term Memory (xLSTM)

The Extended Long Short-Term Memory (xLSTM) [4] was recently developed to combine the efficiency of recurrent models with the performance of modern Transformers. It introduces two new types of blocks: the sLSTM and the mLSTM.

sLSTM and mLSTM

The sLSTM, or scalar LSTM, improves the original design by using exponential gating. In traditional models, gates use functions that can limit how quickly a model learns or forgets. By using exponential functions, the sLSTM can perform more aggressive updates, allowing it to overwrite or prioritize information with much higher precision. To prevent the numbers from becoming too large and unstable, the sLSTM uses a normalization state. This stabilizer keeps track of the exponential weights and ensures the memory remains mathematically balanced even during long sequences.

The mLSTM, or matrix LSTM, significantly increases memory capacity by replacing the hidden vector with a matrix-valued memory. This allows the model to store complex relationships between observations, similar to how a Transformer stores information, but within a recurrent structure. A key feature of the mLSTM is its dual form. This means the model can be calculated in two different ways: it can work step-by-step for fast interaction with an environment, or it can be calculated in a parallel mode for fast training on large datasets.

This parallel processing is made possible by the removal of memory mixing. In older LSTMs, the different parts of the hidden state were connected to each other in a way that forced the computer to calculate them one after another. By removing these internal dependencies, the mLSTM allows the model to be trained across many GPUs simultaneously, much like a Transformer.

Scaling and the 7B Model

The development of the xLSTM-7B model [3] was a significant milestone, proving that recurrent architectures can scale to the same size as modern Transformers like Llama [26] or Gemma [8]. The primary motivation was to show that a model could maintain the semantic capacity. This just means that the model has the ability to recognize complex patterns and relationships in data—of a large-scale foundation model while keeping the high inference speed and low memory usage of an LSTM.

To achieve this scale, the model relies on the parallel training capability of the mLSTM block. In traditional LSTMs, each step must wait for the previous one to finish, making training on large clusters very slow. By removing hidden-to-hidden recurrence, also known as memory mixing, the xLSTM can be computed in a parallel form during training. In older LSTMs, the components of memory were mixed together at every step, creating a dependency that forced sequential calculation. By decoupling these connections, the mLSTM allows the model to process trillions of tokens across thousands of GPUs, a feat that was previously only possible for Transformers.

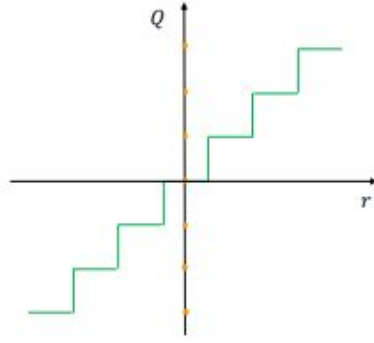


Figure 2.3: In Uniform Quantization, real values (r) from a continuous domain are mapped to discrete, equidistant levels (orange bullets) in the quantized domain (Q) [10].

The 7B version specifically optimizes the architecture for token throughput. By using a structure where the memory operations happen efficiently within the model’s main embedding dimensions, the authors were able to increase processing speeds significantly compared to earlier versions. For the purposes of this thesis, using a 7B-parameter memory module means the SHELm agent has access to a much higher representational capacity for the semantic tokens it receives.

This capacity allows the agent to build more detailed and accurate models of its environment. Because the xLSTM maintains linear $\mathcal{O}(N)$ complexity, the agent can handle long-duration episodes without the performance drops or memory crashes that often affect Transformer-based models, which require an ever-increasing amount of memory as the history grows.

2.4 Large-Scale Model Optimization

While 7-billion-parameter models like the xLSTM-7B [3] offer significant representational capacity, they present substantial computational challenges. To make these models feasible for research and deployment on hardware with limited resources, optimization techniques are required to reduce the memory footprint without severely compromising performance.

2.4.1 Quantization

Quantization is a model compression technique designed to alleviate memory and computational bottlenecks by reducing the bit-width of model parameters [7, 10]. While large-scale language models are commonly distributed in high-precision floating-point formats such as standard 32-bit (FP32) or 16-bit Brain Floating Point (BF16) [14], performing inference at this precision is often prohibited by the memory capacity of consumer-grade hardware. The mapping of real values from a continuous domain to discrete levels is shown in Figure 2.3.

For instance, a 7-billion parameter model in FP32 requires approximately 28 GB of Video Random Access Memory (VRAM), which exceeds the capacity of widespread GPUs. In this work, we employ 8-bit integer (Int8) quantization for the xLSTM recurrence backbone. This technique maps continuous floating-point weights to a discrete set of

2^8 (256) representable values. Compared to a full-precision FP32 baseline, this yields a theoretical $4\times$ reduction in memory footprint, allowing the 7B model to reside within approximately 7.5 GB of VRAM.

The decision to utilize 8-bit quantization over more aggressive 4-bit techniques (e.g., NF4) was driven by hardware-specific latency constraints. Empirical testing revealed that on such hardware, 8-bit quantization provides the optimal trade-off: it sufficiently compresses the model to fit in memory while maintaining high-throughput inference speeds critical for the reinforcement learning inner loop.

Chapter 3

Experiments

To evaluate the efficacy of the proposed Semantic History Compression (SHELM) approach using the xLSTM-7B architecture, we selected two distinct reinforcement learning environments: **MiniGrid-Memory** and **PsychLab Continuous Recognition**. These environments were chosen specifically for their heavy reliance on the agent’s ability to store and recall information over time, making them ideal benchmarks for evaluating memory modules as established in previous work [19].

To ensure our results are directly comparable to the baselines established in the original SHELM study, we utilize the same hyperparameters and training configurations as Paischer et al. [19]. The agents are trained using Proximal Policy Optimization (PPO) [24]. The training duration is limited to 2 million interaction steps for the MiniGrid environment and 6 million steps for the visually complex PsychLab environment. We set the number of parallel environments to 8 to respect VRAM constraints, accumulating gradients over 128-step rollouts to maintain a sufficient batch size for stable policy updates.

All experiments reported in this chapter were conducted on a single NVIDIA GeForce GTX 1080 Ti with 11 GB of VRAM, an AMD Ryzen 5 2600X CPU, and 16 GB of system RAM. This constraint serves as a practical stress test for the 8-bit quantization and optimization techniques described in Section 2.4.

3.1 Methodology - Architecture Integration

The core of our contribution is the replacement of the trainable, randomly initialized TrXL module (250M parameters) with a frozen, pre-trained xLSTM-7B model (7000M parameters). Unlike the Transformer-XL, which relies on a sliding window of cached memory segments, the xLSTM architecture maintains a continuous recurrent hidden state.

Recurrent State Management We developed an XLSTMAadapter module to bridge the interface between the SHELM environment interaction loop and the HuggingFace-style xLSTM implementation. A critical challenge in Reinforcement Learning (RL) with recurrent models is the handling of episode boundaries. In batched environments, different trajectories terminate at different times. To address this, we implemented a recursive state reset mechanism that selectively zeros out the hidden states only for the environments that signaled a `done` flag, ensuring that the model’s memory does not bleed across independent episodes.

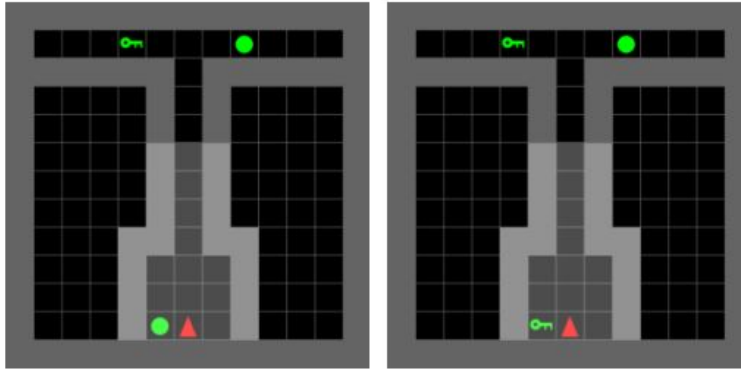


Figure 3.1: Two sample environments of the MiniGrid - Memory task, where the agent has to differentiate between two objects [5].

3.2 MiniGrid-Memory

The first environment is the MiniGrid-Memory task (specifically ‘MiniGrid-MemoryS11-v0’) from the Gymnasium-MiniGrid suite [5]. This is a partially observable grid world designed to test an agent’s working memory. Two sample environments of the task are shown in 3.1.

In this task, the agent begins in a small room containing a specific object, such as a colored ball or key. This object serves as the "cue." The agent must observe this cue and then navigate through a corridor to reach a decision point. At the end of the corridor, the agent is presented with two objects: one that matches the initial cue and one that does not. To receive a reward, the agent must select the matching object.

The difficulty lies in the partial observability; the cue is only visible at the very beginning of the episode. While traversing the corridor or facing the wall, the agent cannot see the cue. Therefore, the agent must encode the cue’s information (e.g., "green ball") into its internal memory state and retain it throughout the navigation phase to make the correct decision at the end. If the agent’s memory fails or is overwritten during navigation, it will be forced to guess, resulting in a significantly lower success rate. Without a working memory the reward reached would come out to about 0.5.

3.3 PsychLab - Continuous Recognition

The second environment is the Continuous Recognition task from the PsychLab suite [17]. Unlike the symbolic grid world of MiniGrid, PsychLab is a first-person 3D environment built on top of the DeepMind Lab platform [2]. It is designed to replicate standard cognitive psychology experiments used to measure memory retention in humans and animals.

In the Continuous Recognition task, the agent is presented with a stream of visual stimuli (images of objects) in a sequential manner. For each object presented, the agent must decide whether it has seen this specific object before within the current episode by swiping on a virtual screen. An agent without a functional memory component would reach a reward of approximately 30 with random swipes. 3.2 shows some different objects with their respective observations.



Figure 3.2: PsychLab Continuous Recognition task with a sequence of objects [19].

This task is significantly more demanding than MiniGrid for two reasons. First, the observations are high-dimensional pixel inputs rather than simple grid states, requiring the CLIP encoder to extract meaningful semantic features, as shown in Paischer et al. Second, the memory load increases continuously as the episode progresses; the agent must maintain an ever-growing history of seen objects to correctly identify novel versus repeated stimuli. This setup rigorously tests the capacity of the xLSTM-7B memory module to compress and retrieve specific visual instances from a long temporal context.

Chapter 4

Results

4.1 Minigrid-Memory Results

In this section, we present the empirical evaluation of the SHELm-xLSTM integration. Our analysis focuses on two key aspects: first, the optimization stability required to fine-tune the 7B parameter backbone, and second, a comparative benchmark against established Transformer-XL (TrXL) baselines from [12, 19].

4.1.1 Optimization Stability Analysis

Figure 4.1 illustrates the learning trajectories of our two xLSTM configurations.



Figure 4.1: Comparison of the proposed low-LR configuration (5×10^{-5}) and the standard baseline (5×10^{-4}).

The standard learning rate (5×10^{-4}) proved too aggressive for the frozen backbone; while the agent learned a basic policy (≈ 0.5 reward), it struggled to refine its behavior, only reaching 0.8 reward after 1M steps and never achieving full mastery. In contrast, our proposed xLSTM agent (Low LR: 5×10^{-5}) demonstrated rapid and stable convergence, reaching a solved state (Reward ≈ 1.0) in just 500,000 timesteps. This confirms that a restricted learning rate regime is critical for fine-tuning large-scale pre-trained models in RL.

4.1.2 Comparative Benchmark

To contextualize this performance, Figure 4.2 compares our best xLSTM-7B agent against the Transformer-XL baselines.

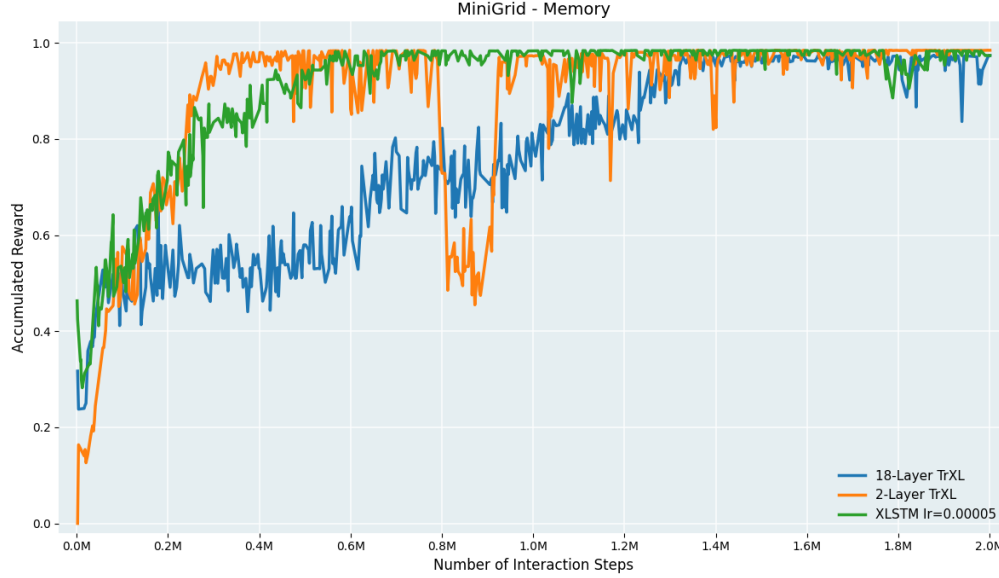


Figure 4.2: The xLSTM-7B agent (Ours) vs. standard Transformer-XL baselines [12, 19].

1. **vs. Standard SHELM (18-Layer TrXL):** Our xLSTM agent converged to the solved state nearly 3x faster than the 18-layer Transformer baseline (500k steps vs. 1.4M steps), demonstrating a significant improvement in sample efficiency compared to the deep baseline.
2. **vs. Fast SHELM (2-Layer TrXL):** While the lightweight 2-layer model learned quickly (reaching 1.0 reward at ≈ 300 k steps), it exhibited significant instability during training, frequently dropping below 0.8 reward. Our xLSTM agent, by contrast, maintained a stable performance once the solved state was reached.

Conclusion These results indicate that the xLSTM-7B model effectively balances sample efficiency and training stability. Although it does not converge as rapidly as the shallow 2-layer baseline (500k vs. 300k steps), it avoids the severe performance drops observed in that smaller architecture. Compared to the 18-layer baseline, the xLSTM agent provides the same stable retention of the learned policy while requiring substantially fewer environment interactions to converge. Consequently, the xLSTM architecture demonstrates its viability as a memory backbone for Semantic RL, provided the necessary computational resources are available.

4.1.3 Effect of Layer Normalization on History Compression

To assess potential improvements in signal stabilization, we conducted an ablation study applying Layer Normalization (LayerNorm) [1] to the compressed history representation. The objective was to normalize the historical signal relative to the current observation; however, this addition did not yield the anticipated performance gains.



Figure 4.3: The average reward over 2M Timesteps for the two different runs with LayerNorm against our best run without it.

Figure 4.3 displays the learning trajectories of the LayerNorm configurations alongside our best non-normalized baseline. When trained with the standard learning rate, the agent failed to converge, with its performance heavily fluctuating around a suboptimal reward of 0.5. Adjusting the learning rate to our proposed lower regime stabilized the training process; however, the resulting policy performed similarly to—and occasionally marginally worse than—our proposed xLSTM agent without LayerNorm. Due to the lack of tangible benefits and the slight performance degradation, LayerNorm was omitted from our final proposed architecture.

4.2 PsychLab - Continuous Recognition Results

The PsychLab Continuous Recognition task serves as a stress test for long-horizon memory retention. Unlike MiniGrid, where the memory must be held for a fixed duration, PsychLab requires the agent to continuously recognize novel vs. familiar objects over extended episodes.

4.2.1 Learning Performance

During the initial phase of training (up to approximately 1 million timesteps), both architectures exhibit similar learning trajectories. In this early stage, the agents are likely adapting to the fundamental mechanics of the environment and the basic visual feature extraction required by the task, resulting in comparable baseline performance.

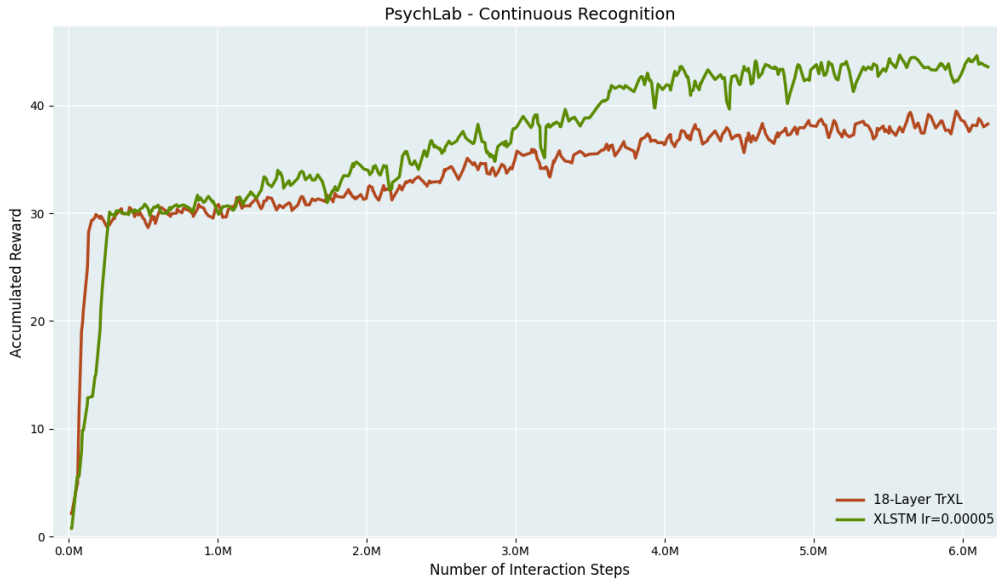


Figure 4.4: Average return on PsychLab Continuous Recognition of the xLSTM-7B model compared against the TrXL baseline over 6 million timesteps.

The results shown in Figure 4.4 indicate that as training progresses beyond the 1 million timestep mark, a clear divergence emerges. The xLSTM-7B agent begins to gradually outperform the original 18-Layer TrXL configuration. By the end of the 6 million timestep evaluation, the xLSTM-7B achieves a final average reward of approximately 43, whereas the Transformer-XL baseline stops at roughly 39.

Wall-Clock Runtime

While the xLSTM-7B architecture demonstrates a higher final performance ceiling, it is important to contextualize this with the computational overhead required to process a 7-billion-parameter backbone. Table 4.1 outlines the practical wall-clock runtimes and average step latencies required for the models to complete the 6 million timesteps. The time taken per parameter update is substantially higher for the xLSTM, resulting in a significantly longer total training duration compared to the 18-Layer TrXL baseline.

Table 4.1: Comparison of practical wall-clock runtimes for full training on the PsychLab - Continuous Recognition environment (6,000,000 timesteps).

Model Architecture	Parameters (Approx.)	Total Wall-Clock Runtime	Time per Step
Transformer-XL (18-Layer)	250M	1d 20h 37m 17s	≈ 26.8 ms
xLSTM-7B (Quantized)	7000M	5d 2h 11m 49s	≈ 73.3 ms

Inference Benchmark

However, measuring raw total training time convolutes the model’s pure forward pass latency with the environment rendering overhead and backpropagation calculations. To explicitly isolate the computational cost of the forward pass latency, a standardized inference benchmark was conducted outside of the Reinforcement Learning loop simulating sequential environment rollouts ($N = 2048$). The results of this isolated test are presented in Table 4.2.

The isolated evaluation reveals that the xLSTM forward pass was empirically $\sim 9\times$ slower per step. Crucially, this latency penalty is radically smaller than the architectural size difference between the two configurations, as the xLSTM-7B is approximately $28\times$ larger in parameter count (7B vs. 250M) than the baseline.

Table 4.2: Isolated inference throughput benchmark ($N = 2048$ sequential forward passes).

Model Architecture	Throughput	Time per Step
Transformer-XL (18-Layer)	18.49 steps/s	54.1 ms
xLSTM-7B (8-Bit Quantized)	2.06 steps/s	484.8 ms

Chapter 5

Discussion

The primary objective of this thesis was to evaluate whether the computational and structural bottlenecks of the Semantic History Compression (SHELM) architecture could be resolved by replacing the standard Transformer-XL (TrXL) memory module with the xLSTM architecture. The empirical results in Section 4 demonstrate that integrating the xLSTM model significantly enhances sample efficiency compared to deep baselines and improves stability compared to shallow baselines in memory-dependent tasks. However, utilizing a memory backbone with 7 billion parameters introduces new optimization dynamics and practical hardware limitations that must be carefully managed.

5.1 Interpretation of Performance Gains

The evaluation on the MiniGrid-Memory environment highlights a stark contrast in sample efficiency and learning dynamics between the xLSTM-7B backbone and the traditional TrXL baselines. Our proposed xLSTM agent achieved convergence to a solved state in approximately 500,000 interaction steps. In comparison, the standard 18-layer TrXL required nearly 1.4 million steps to reach the same performance threshold, while the lightweight 2-layer TrXL converged rapidly but remained highly unstable. Given that both the 18-layer TrXL and the xLSTM-7B are pre-trained architectures, the 3 $\times$ acceleration in learning speed cannot be attributed simply to pre-training versus training from scratch, nor can it be purely a function of model capacity—otherwise, the shallow 2-layer model would not be the fastest to converge. Instead, these differences are better understood through the lens of representation decoding and structural alignment in Reinforcement Learning.

The 18-layer TrXL represents a significant optimization bottleneck: its deep, attention-based representations are highly complex, requiring the RL policy network a vast number of interactions to decode these features into viable navigational actions. Conversely, the 2-layer TrXL produces shallower, surface-level features that are easier for the RL agent to map to actions, but lacks the structural depth necessary to maintain robust, long-horizon memory, leading to severe instability.

The xLSTM-7B agent effectively bridges this gap through structural alignment. Thanks to its massive scale and modern pre-training, the features extracted by the xLSTM are of exceptional quality and highly structured. More importantly, its continuous recurrent memory mechanics, unlike the discontinuous, block-based context limits of a Transformer, align naturally with the sequential nature of RL environments. This allows the RL policy to decode actionable signals faster than with the deep TrXL, while its profound depth and continuous gating mechanisms provide the long-term stability that the 2-layer model fundamentally lacks. Furthermore, the results from the PsychLab Continuous Recognition task validate the xLSTM’s extended capability in long-horizon memory tasks. While both

models showed similar performance during the initially simplistic visual-feature adaptation phase up to 1 million steps, the xLSTM eventually broke away to establish a higher performance ceiling.

5.2 Asymptotic vs. Practical Runtime

A central premise of this work was transitioning from a memory context model bounded by sliding-window attention to the theoretically superior, constant-time state updates of the xLSTM. However, during practical experiments, the xLSTM-7B exhibited substantially longer wall-clock runtimes per episode than the baseline TrXL (Table 4.1). This discrepancy arises from the massive difference in parameter counts masking the true architectural efficiency. In Big- $\mathcal{O}$ notation, constant factors are abstracted away. Yet, in physical hardware deployments, the total iterative step computation time T depends heavily on the total number of trainable parameters P . During continuous iterative inference in RL rollouts, Transformer-XL must calculate attention dot products against past representations stored in an explicit memory cache of sequence length N . Therefore, the baseline TrXL memory module (approximately 250 million parameters) scales per step as:

$$T_{\text{TrXL}} \approx 250M \cdot \mathcal{O}(N) \quad (5.1)$$

In contrast, the xLSTM architecture performs an $\mathcal{O}(1)$ iterative update by mathematically modifying its compressed hidden matrix state at each step, removing the necessity to reference a growing sequence length. However, the model contains over 7 billion parameters, introducing an approximate 28-fold constant computation penalty:

$$T_{\text{xLSTM}} \approx 7000M \cdot \mathcal{O}(1) \quad (5.2)$$

Because the xLSTM is roughly 28 times larger, it incurs a massive baseline computational penalty. However, the isolated inference benchmark on PsychLab (Table 4.2) completely validates the underlying theoretical efficiency of the xLSTM architecture. While the xLSTM forward pass was empirically $\sim 9\times$ slower per step than TrXL, this performance penalty is substantially less severe than the $28\times$ architectural scaling difference.

This non-linear scaling penalty proves the efficiency of the $\mathcal{O}(1)$ execution. For the TrXL baseline, evaluating $N = 511$ past tokens via attention acting as a heavy $\mathcal{O}(N)$ computational multiplier that offsets its smaller parameter count. The observed $9\times$ computational slowdown is therefore not an inefficiency of the xLSTM architecture itself, but rather an artifact of the sheer magnitude of evaluating a 7-billion-parameter array.

Furthermore, fitting a model of this scale required executing the network in 8-bit precision (Int8) to operate within standard GPU VRAM constraints. This quantization introduces a strict hardware latency penalty, as the GPU must continuously dequantize the low-precision weights to 16-bit floats prior to every matrix multiplication, heavily bounding the execution by memory bandwidth. By comparison, the 250M-parameter TrXL executes in hardware-optimized, native 32-bit floats.

Ultimately, these empirical measurements confirm that substituting an $\mathcal{O}(N)$ attention stepping mechanism with an $\mathcal{O}(1)$ xLSTM memory state creates massive computational scaling advantages per parameter. If both architectures were scaled to matching parameter volumes and evaluated in equivalent precision environments, the isolated scaling differential strongly implies the xLSTM would exhibit vastly superior throughput ceilings.

5.3 Optimization Dynamics and Stability

Our experiments demonstrated that the optimization schedule was the primary factor in maintaining training stability when fine-tuning atop a massive backbone. When using the standard baseline learning rate of 5×10^{-4} , the agent experienced catastrophic gradient updates that destabilized the trainable CNN components. By strategically reducing the learning rate to 5×10^{-5} , the agent was able to stably align its trainable, lower-dimensional visual encoders with the frozen 4096-dimensional representations of the xLSTM backbone. This dynamic highlights an important consideration for Semantic RL: while massive frozen backbones provide a "stable anchor" of pre-trained knowledge, they simultaneously require a regulated optimization strategy to prevent projecting noisy gradients into the high-dimensional latent space, which causes policy divergence.

Additionally, our ablation study on Layer Normalization revealed that attempting to normalize the compressed history representation relative to the current observation was counterproductive. Rather than stabilizing the signal, LayerNorm caused the agent to fail to converge under standard learning rates and occasionally degraded performance even under the lower, stabilized learning rate regime. This suggests that the raw, unnormalized historical signal from the xLSTM backbone is already optimally scaled for the downstream trainable components, and forcing normalization disrupts the learned representations. Consequently, preserving the raw signal without additional normalization layers proved essential for stable optimization.

5.4 Limitations and Future Work

While this thesis successfully demonstrated the integration and potential of xLSTM within the SHELM architecture, several limitations must be acknowledged to contextualize the findings.

1. **System-Wide Hardware and Quantization Constraints:** The experiments in this thesis were severely constrained by the designated hardware setup—specifically, execution on a single NVIDIA GTX 1080 Ti GPU (11 GB VRAM), an AMD Ryzen 5 2600X CPU, and 16 GB of system RAM. Fitting a 7B model locally is impossible in standard precision on this hardware. Therefore, 8-bit uniform quantization was mandatory. While quantization enabled execution, it introduced representation degradation. Furthermore, the massive runtime differences are not solely due to GPU compute speed; they are symptomatic of a system-wide bottleneck. Managing the multi-gigabyte memory footprint of the xLSTM alongside the RL environment state severely taxed the limited 16 GB of system RAM, while continuous cross-communication and orchestration between the CPU and GPU heavily saturated the PCIe bandwidth. The true, unquantized performance and dynamics of the xLSTM-7B model on high-end enterprise hardware remain unassessed.
2. **Statistical Variance (Single-Seed Runs):** Due to the extreme wall-clock runtimes caused by the massive parameter count and hardware limitations (over 122 hours for a single xLSTM run as shown in the table 4.1), the documented results are based on single-seed runs. Reinforcement learning is notoriously sensitive to initialization, and best practices dictate averaging performance across multiple random seeds (e.g., 3 to 5 runs) to calculate reliable confidence intervals. While the single-seed curves demonstrate clear and promising performance trends, absolute statistical robustness cannot be definitively claimed without broader multi-seed verification.

3. **Entangled Variables (Scale vs. Architecture):** As noted in Section 5.2, the current experimental setup evaluates a 7-billion-parameter recurrent model against a 250-million-parameter attention model. This scale gap makes it impossible to definitively declare whether the efficiency gains are native to the xLSTM architecture itself or simply a byproduct of parameter scale.

Future research should aim to isolate these variables—either by evaluating heavily scaled-down xLSTM model variants or scaling up TrXL baselines to match. Notably, our choice of the 7B xLSTM model was constrained by availability, as no smaller or alternative pre-trained xLSTM variants are currently publicly accessible. Additionally, benchmarking the architecture on environments requiring order-of-magnitude longer sequence horizons ($N > 10,000$) would provide the necessary conditions to observe the true hardware-level benefits of xLSTM’s linear computational scaling.

Chapter 6

Conclusion and Outlook

In this thesis, we aimed to enhance the scalability and memory capacity of the Semantic History Compression (SHELM) framework [19] by replacing the standard Transformer-XL backbone with the recently introduced xLSTM-7B architecture [4]. This transition was motivated by the inherent $\mathcal{O}(N^2)$ complexity of Transformers, which creates a computational and memory bottleneck as the agent’s history grows. By utilizing a recurrent model with linear scaling, we sought to maintain a high-capacity semantic memory that remains stable over long-duration episodes.

The empirical findings of this work demonstrate that replacing the Transformer-XL memory with a quantized xLSTM-7B backbone is not only feasible under constrained hardware conditions, but can also provide substantial benefits in sample efficiency and optimization stability. On the MiniGrid-Memory benchmark, the xLSTM-based SHELM agent converged significantly faster than the standard 18-layer Transformer-XL baseline while maintaining stable performance once the task was solved. This suggests that large-scale pre-trained recurrent models can serve as powerful semantic memory anchors in reinforcement learning, enabling rapid adaptation with comparatively few environment interactions.

On the more demanding PsychLab Continuous Recognition task, while the xLSTM-7B backbone initially showed performance comparable to the Transformer-XL baseline, it ultimately demonstrated superior long-term memory retention. Progressively outperforming the attention-based architecture in the visually complex setting it successfully proved that a linear-scaling recurrent model can effectively sustain long-horizon memory in high-dimensional environments without the hard context-window truncations of standard Transformers. Importantly, these results were achieved using 8-bit quantization and consumer-grade hardware, highlighting the practical viability of integrating foundation-scale recurrent models into RL pipelines.

At the same time, the study revealed important trade-offs. Although xLSTM provides asymptotically linear complexity in sequence length, the massive parameter count of the 7B model introduces significant constant-factor overheads that dominate runtime in short- to medium-horizon tasks. Thus, the theoretical advantage of $\mathcal{O}(N)$ scaling only becomes practically relevant at sufficiently long context lengths. Furthermore, the observed performance gains are entangled with model scale, leaving open the question of how much improvement stems from architectural differences versus sheer parameter capacity.

Overall, this thesis shows that recurrent large language models are a compelling alternative to attention-based memory modules in Semantic History Compression. The integration of xLSTM into SHELM demonstrates that linear-time sequence models can match or exceed Transformer-based approaches in memory-dependent reinforcement learning while offering improved stability characteristics under careful optimization.

Looking forward, several research directions emerge. First, controlled scaling studies are required to disentangle architectural effects from parameter count by comparing similarly sized Transformer and xLSTM models. Second, future experiments should evaluate the architecture in environments with substantially longer temporal dependencies, where quadratic attention costs become computational bottlenecks, allowing the linear time complexity of recurrent models to demonstrate significant efficiency gains. Third, deploying unquantized or mixed-precision variants of xLSTM on modern high-memory hardware would allow assessment of its full representational capacity without compression artifacts. Finally, hybrid approaches that combine semantic retrieval with structured recurrent state updates—or that selectively fine-tune portions of the backbone—may further improve alignment between pre-trained knowledge and task-specific memory demands.

In conclusion, this work provides evidence that large-scale recurrent language models such as xLSTM can function as efficient, high-capacity memory modules for partially observable reinforcement learning. By bridging semantic compression and scalable recurrence, it contributes to the broader goal of building reinforcement learning agents capable of reasoning over extended histories without incurring prohibitive computational costs.

Bibliography

- [1] Jimmy Lei Ba, Ryan Kiros, and Geoffrey E Hinton. 2016. Layer Normalization. *arXiv preprint arXiv:1607.06450*.
- [2] Charles Beattie, Joel Z. Leibo, Denis Teplyashin, Tom Ward, Marcus Wainwright, Heinrich Küttler, Andrew Lefrancq, Simon Green, Víctor Valdés, Amir Sadik, Julian Schrittwieser, Keith Anderson, Sarah York, Max Cant, Adam Cain, Adrian Bolton, Stephen Gaffney, Helen King, Demis Hassabis, Shane Legg, and Stig Petersen. 2016. DeepMind Lab. <https://arxiv.org/abs/1612.03801>.
- [3] Maximilian Beck, Korbinian Pöppel, Phillip Lippe, Richard Kurle, Patrick M. Blies, Günter Klambauer, Sebastian Böck, and Sepp Hochreiter. 2025. xLSTM 7B: A Recurrent LLM for Fast and Efficient Inference. (2025). arXiv: 2503.13427 [cs.LG]. <https://arxiv.org/abs/2503.13427>.
- [4] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. 2024. xLSTM: Extended Long Short-Term Memory. *arXiv preprint arXiv:2405.04517*.
- [5] Maxime Chevalier-Boisvert, Lucas Willems, and Suman Pal. 2018. Minimalistic Gridworld Environment for Gymnasium. <https://github.com/Farama-Foundation/MiniGrid>. (2018).
- [6] Zihang Dai et al. 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. In *ACL*.
- [7] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *arXiv preprint arXiv:2208.07339*.
- [8] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295*.
- [9] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. 2000. Learning to Forget: Continual Prediction with LSTM. *Neural Computation*, 12, 10, 2451–2471. DOI: 10.1162/089976600300015015.
- [10] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv preprint arXiv:2103.13630*.
- [11] Danijar Hafner et al. 2020. Dream to Control: Learning Behaviors by Latent Imagination. In *ICLR*.
- [12] Dominik Heindl. 2024. *Speeding up Semantic History Compression in Reinforcement Learning*. Master’s thesis. Johannes Kepler University Linz.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation*, 9, 8, 1735–1780.

- [14] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. 2017. In-Datcenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. ACM. DOI: 10.1145/3079856.3080246.
- [15] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101, 1-2, 99–134.
- [16] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86, 11, 2278–2324.
- [17] Joel Z Leibo, Cyprien de Masson d’Autume, Daniel Zoran, David Amos, Charles Beattie, Keith Anderson, Antonio García Castañeda, Manuel Sanchez, Simon Green, Audrunas Gruslys, et al. 2018. Psychlab: A psychology laboratory for deep reinforcement learning agents. *arXiv preprint arXiv:1801.08116*.
- [18] Michael List. 2024. *Distillation for Efficient History Compression in Reinforcement Learning*. Master’s thesis. Johannes Kepler University Linz.
- [19] Fabian Paischer et al. 2023. Semantic History Compression for Reinforcement Learning. *arXiv preprint arXiv:2312.11264*.
- [20] Fabian Paischer, Thomas Adler, Vihang Patil, Angela Bitto-Nemling, Markus Holzleitner, and Sepp Hochreiter. 2022. History Compression via Language Models in Reinforcement Learning. In *ICML*.
- [21] Fabian Paischer, Thomas Adler, Andreas Radler, Markus Hofmarcher, and Sepp Hochreiter. 2022. Toward Semantic History Compression for Reinforcement Learning. In *Second Workshop on Language and Reinforcement Learning, NeurIPS*.
- [22] Alec Radford et al. 2021. Learning Transferable Visual Models from Natural Language Supervision. In *ICML*.
- [23] Jürgen Schmidhuber. 1992. Learning complex, extended sequences using the principle of history compression. *Neural Computation*, 4, 2, 234–242.
- [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- [25] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement Learning: An Introduction*. MIT press.
- [26] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*.
- [27] Ashish Vaswani et al. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.

List of Figures

2.1	Illustration of the standard Reinforcement Learning interaction loop [25].	4
2.2	Overview of the Semantic HELM (SHELM) architecture [19].	6
2.3	In Uniform Quantization, real values (r) from a continuous domain are mapped to discrete, equidistant levels (orange bullets) in the quantized domain (Q) [10].	9
3.1	Two sample environments of the Minigrid - Memory task, where the agent has to differentiate between two objects [5].	12
3.2	PsychLab Continuous Recognition task with a sequence of objects [19]. . .	13
4.1	Comparison of the proposed low-LR configuration (5×10^{-5}) and the standard baseline (5×10^{-4}).	14
4.2	The xLSTM-7B agent (Ours) vs. standard Transformer-XL baselines [12, 19].	15
4.3	The average reward over 2M Timesteps for the two different runs with Layernorm against our best run without it.	16
4.4	Average return on PsychLab Continuous Recognition of the xLSTM-7B model compared against the TrXL baseline over 6 million timesteps. . . .	17

List of Tables

- 4.1 Comparison of practical wall-clock runtimes for full training on the PsychLab - Continuous Recognition environment (6,000,000 timesteps). . . . 17
- 4.2 Isolated inference throughput benchmark ($N = 2048$ sequential forward passes). 18